



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

EXECUTIVE SUMMARY OF THE MULTIDISCIPLINARY PROJECT

Have you ever seen an Alpaka doing molecular docking?

GRADUATE PROGRAM IN HIGH PERFORMANCE COMPUTING ENGINEERING

Author: OLEG NEDINA (10905938)

Advisor: LEONARDO BELTRAME

Co-advisors: PROF. PALERMO, PROF. GADIOLI

Academic year: 2025-2026

Abstract

This report documents the porting of a complex application to address the issue of hardware vendor lock-in. This was achieved by transitioning the codebase to Alpaka, an open-source C++ performance portability framework.

The target application is MuDock, a molecular docking tool developed by the Polimi team. MuDock relies on various backends depending on the target hardware. The core of this work involved implementing a unified Alpaka backend from scratch, comparing its performance against the existing native backends, and analyzing the trade-off between portability and computational efficiency.

All tests and performance metrics were collected on the *Accento* cluster, provided by Politecnico di Milano. The evaluation was carried out using three datasets for the GPU and one for the CPU, with full technical details provided in the following sections.

The outcome of this integration is highly promising. Starting from a naive baseline and progressively optimizing the code, the Alpaka implementation achieved performance identical to the native GPU backends, alongside comparable results on the CPU.

This proves that it is possible to maintain ex-

tremely high performance using a single `.cpp` source file across different machines and accelerators, without relinquishing code portability.

Although, significant milestones have been reached, further optimizations are possible; final considerations and potential improvements are discussed at the end of this document.

The main quantitative results documented in this report are:

- On **Single-instance GPU**: Alpaka CUDA achieves **308.2 Evals/s** compared to **304.9 Evals/s** for native CUDA (+1.1%) (see Section 5.4).
- On **Serial CPU**: Alpaka introduces **zero overhead** compared to native C++ (**34.8** vs **34.9 Evals/s**) (see Section 5.1).

Note: While currently adopted by CERN for projects like ROOT, the Alpaka framework [9][10] was originally developed within the PIconGPU project at HZDR in Dresden, not at CERN. For the purpose of a paper review, this distinction is needed to properly position the contribution.

Table 1: Comparison of portability frameworks.

Framework	Vendor	Model	Pros	Cons
CUDA	NVIDIA	Proprietary	Max optimization on NVIDIA HW	Zero portability on AMD/Intel
HIP	AMD	Proprietary (CUDA-like)	Almost automatic porting from CUDA	Tied to the AMD ecosystem
SYCL / DPC++	Intel/Khronos	Open Standard	Portable, modern C++	Mature toolchain mostly on Intel
Kokkos	Sandia NL	Header-only C++	Excellent OpenMP/CUDA support	Verbose API, steep learning curve
Alpaka [8]	HZDR	Header-only C++	Zero-overhead, used in HPC	Smaller community than Kokkos

1. Introduction

The portability problem stems from the necessity of writing distinct code for different hardware architectures, which introduces significant practical challenges in terms of software maintenance and future improvements. A larger codebase inherently increases the risk of introducing errors, especially considering that developers must make different design choices for each specific backend.

A practical example is the use of graphics accelerators. Typically, selecting a specific graphics card also mandates the use of a proprietary programming language such as CUDA for NVIDIA or HIP for AMD. This scenario translates directly into vendor lock-in, severely restricting the ability to experiment and switch hardware platforms.

Therefore, this project was born with a clear question in mind:

"Can this be done differently?"

Following an initial market analysis of the various portability frameworks and libraries currently available, it was decided to proceed with Alpaka. Table 1 lists the evaluated frameworks along with the rationale behind this choice.

The three main reasons for proceeding with Alpaka are:

1. **Zero-overhead philosophy:** Alpaka relies entirely on template metaprogramming.

When compiled for a specific target, it generates code that is practically indistinguishable from hand-written native code in the reference language. Notably, there is no runtime dispatch layer.

2. **Production-proven:** Alpaka is the core library behind **PICongPU** [8][3], a highly performant HPC plasma simulator already deployed on Tier-0 systems.
3. **Direct compatibility with MuDock:** MuDock [1] is a mini-app that features well-defined and isolated kernels, making it an ideal candidate for seamless integration.

This report documents the integration process in detail. The remainder of this document is structured as follows: Section 2 introduces the key terminology, the MuDock application, and the parallel frameworks utilized. Section 3 details the methodology, the work breakdown, and the validation tests performed to ensure correctness. Section 4 outlines the full experimental setup, including the hardware node, datasets, and profiling suite used for performance evaluation. Section 5 presents the quantitative results across both CPU and GPU backends, analyzing scalability, host resources, and micro-architectural metrics. Finally, Section 6 is divided into two distinct parts: the first formally concludes the report by evaluating the overall results against

the initial objectives, while the second provides a retrospective discussion recounting the practical engineering journey, detailing the architectural challenges and lessons learned with Alpaka, and outlining future work.

2. Terminology, Tools, and Frameworks

This section is dedicated to introducing all the key terms, tools, and frameworks utilized to carry out this work. It provides a self-contained reference covering the domain-specific terminology (Section 2.1), the target application and its internal architecture (Section 2.2), and the parallel programming frameworks alongside the software dependency stack (Section 2.3).

2.1. Important Terms

A brief definition of all the key terms used throughout this report is provided below to avoid any confusion for the reader.

Backend Target hardware/software infrastructure on which the code is compiled and executed (e.g., `Serial`, `OpenMP`, `CUDA`, `HIP`).

Kernel A function executed in parallel by thousands of threads on the accelerator device (GPU or vector CPU). In Alpaka, this is annotated with `ALPAKA_FN_ACC`.

Accelerator (Acc) In Alpaka, the template type representing the execution device. The same kernel receives the specific `Acc` at compile-time, with no runtime dispatch.

Thread / Warp / Block *Thread*: atomic unit of execution. *Block* (`WorkGroup`): a group of threads cooperating via shared memory. *Warp*: 32 threads executing the same instruction in lockstep (SIMT).

Throughput (Evals/s) Number of scoring function evaluations completed per second. Calculated as $(\text{Population} \times \text{Generations}) / \text{Time}$. This is the primary benchmark metric.

Latency (s) Absolute time required to complete a single docking job with a fixed configuration.

Evaluations Total number of scoring function calculations in a job: $\text{Population} \times \text{Generations}$.

Scoring Function / ADT Function evaluating the binding energy of the ligand to

the receptor. In MuDock, it is based on AutoDock4 (ADT4).

LGA *Lamarckian Genetic Algorithm*: a genetic algorithm variant used in AutoDock that combines stochastic evolution with local optimization. MuDock implements the base GA.

Grid Map Pre-calculated 3D grid mapping the receptor-atom interaction energies. It enables fast trilinear interpolation at runtime, avoiding explicit inter-atomic calculations.

TBB / Pipeline Intel Threading Building Blocks. In MuDock, the TBB pipeline manages the asynchronous routing of ligands.

Worker Concurrency unit in MuDock. A worker is currently independent of the TBB pipeline, representing a hybrid asynchronous mechanism under evaluation. While primary data structures are shared among workers to conserve memory, each worker maintains its own execution state and dedicated working buffers.

RSS (Peak RSS) *Peak Resident Set Size*: maximum physical RAM footprint of the process. Measured using `/usr/bin/time -v`.

Occupancy Percentage of active warps on a GPU SM (Streaming Multiprocessor) relative to the theoretical maximum. Low occupancy may indicate excessive register pressure or overly large blocks.

SM Throughput Percentage of the SM's theoretical peak computational throughput that is actually utilized.

AoS / SoA *Array of Structures*: contiguous objects in memory (object-oriented layout). *Structure of Arrays*: each field is stored in a separate array.

FP32 / FP64 32-bit (single precision) and 64-bit (double precision) floating-point numbers. Standard `::sin()` and `::cos()` functions use FP64. Instead, `__sinf()` and `alpaka::math::sin()` operate strictly in FP32.

PTX *Parallel Thread eXecution*: NVIDIA's intermediate assembly language. The `nvcc` compiler generates PTX, which is subsequently translated into `sass` machine code.

HZDR *Helmholtz-Zentrum Dresden-Rossendorf*: the German research institute

where Alpaka was developed within the context of the PIconGPU project.

-march=native GCC/Clang compiler flag enabling the generation of optimized instructions specific to the host CPU (including AVX2, AVX-512 if available).

2.2. Target Application: MuDock

MuDock [1] is an open-source micro-application developed at Politecnico di Milano. Its primary goal is to serve as a controlled testbed for parallelization and optimization techniques in the field of computational molecular docking.

Pipeline Architecture:

The execution flow follows a sequential pipeline: [Ligand Parsing] → [GA Initialization] → [GA Loop: apply → score → iterate] → [Output Results]

The pipeline comprises three main phases, each implemented as a separate kernel:

- **apply** (`geom_transform_alpaka.cpp`): Applies the geometric transformation to the ligand chromosomes, producing the atomic coordinates in the receptor space.
- **adt_score** (`adt_score_alpaka.cpp`): Computes the ADT4 scoring function [10] for each candidate.
- **iterate** (`genetic_alpaka.cpp`): Executes a generation of the Genetic Algorithm [9], updating the chromosome population.

The **ligand-level parallelization** is managed by Intel TBB [14] via a pipeline flow graph. Each node in the pipeline processes a distinct ligand independently.

Build Requirements:

- CMake ≥ 3.22
- GCC ≥ 11 (host compiler)
- NVCC (CUDA Toolkit ≥ 11.0) for the GPU backend
- Alpaka v1.2.0 (header-only)
- Intel oneTBB $\geq 2021.x$
- Boost ≥ 1.78 (`program_options`, `graph`)
- OpenBabel ≥ 3.1 (`.mol2` parsing)

2.3. Frameworks and Libraries

2.3.1 Alpaka

Alpaka [8] is a *header-only* C++ library developed at HZDR in Dresden, Germany. Its goal is to achieve performance portability through Template Metaprogramming: the source code is written once against the Alpaka interface, and

the compiler automatically generates optimized code for the target backend.

Table 2: Alpaka’s execution hierarchy compared to CUDA and OpenMP.

Alpaka Concept	CUDA Equiv.	OpenMP Equiv.
<code>alpaka::Grid</code>	Block Grid	Outer loop
<code>alpaka::Block</code>	Block	Thread group
<code>alpaka::Thread</code>	Thread	Thread
<code>alpaka::Warp</code>	Warp	N/A (noop)
<code>alpaka::Queue</code>	<code>cudaStream_t</code>	Task queue

Key Concept: The Acc template.

In Alpaka, each kernel receives a `TAcc const& acc` object as its first argument. This type is a template parameter resolved entirely at compile-time. If compiled for CUDA, `acc` becomes a CUDA context; if compiled for OpenMP, it becomes an OpenMP context and so on.

```
// Alpaka Kernel:
template<typename TAcc>
ALPAKA_FN_ACC void operator()(TAcc const&
    acc, float* data) {
    auto idx = alpaka::getIdx<alpaka::Grid,
        alpaka::Blocks>(acc)[0u];
    data[idx] = alpaka::math::sin(acc,
        data[idx]);
}

// Equivalent CUDA Kernel:
__global__ void kernel(float* data) {
    int idx = blockIdx.x;
    data[idx] = __sinf(data[idx]);
}
```

These two kernels are functionally equivalent. During compilation for a CUDA backend, the compiler resolves the `TAcc` template and generates identical PTX instructions. The Alpaka syntax `alpaka::getIdx<alpaka::Grid, alpaka::Blocks>(acc)[0u]` directly translates to the hardware `blockIdx.x` register, while mathematical abstractions like `alpaka::math::sin` map directly to the corresponding `__sinf` hardware intrinsics.

Production Example: Alpaka serves as the backbone for PIconGPU [8], utilized for plasma simulations in European Tier-0 supercomputing centers like Frontier (Oak Ridge). The exact same library runs natively on NVIDIA H100, AMD MI300, and Intel/AMD CPUs without altering a single line of application code.

2.3.2 CUDA

CUDA [11] is NVIDIA’s proprietary framework for GPU programming. Its execution model is based on SIMT (*Single Instruction, Multiple Threads*): thousands of threads are grouped into *warps*, which execute the same instruction in lockstep. Threads are further organized hierarchically into *blocks* (sharing local memory) and a *grid* (encompassing all blocks).

2.3.3 OpenMP

OpenMP [13] is an API standard for parallel programming on multi-core CPUs, heavily relying on preprocessor directives (`#pragma omp`). It is worth noting that modern OpenMP standards also support target offloading to GPUs, a feature that Alpaka can leverage via its specific OpenMP target backends, though this aspect was not evaluated in the current work.

2.3.4 Project Dependencies Overview

For completeness, Table 3 summarizes the primary dependencies and libraries utilized in the project.

Table 3: Main project dependencies.

Library	Version	Role
Alpaka [8]	v1.2.0	Portability framework
Intel oneTBB [14]	2021.x	Docking pipeline
OpenBabel [12]	≥ 3.1	.mol2 ligand file parsing
Boost [2]	≥ 1.78	program_options, graph
CUDA Toolkit [11]	12.4.1	GPU backend compilation (NVCC)
CMake [6]	≥ 3.22	Build system, separate backend targets
Spack [4]	latest	Software stack management on Accento

3. Methodology and Implementation

3.1. Work Breakdown

This section describes how the porting work was concretely carried out. Starting from an extremely naive Alpaka implementation (which did not exploit any of Alpaka’s internal APIs), the codebase was progressively expanded and improved step by step. The design choices made and the challenges encountered throughout this process heavily influenced the final implementation.

All files created and modified are listed below, followed by a description of the key optimizations applied to each kernel. The validation methodology is detailed at the end of this section.

The workflow can be conceptually divided into two iterative phases:

1. A primary phase dedicated to validating a single kernel after every code modification using specific test runs (execution flags are listed below).
2. A secondary phase dedicated to performance analysis.

The project aimed to conclude only after reaching a baseline comparable to the native GPU backends. It was subsequently decided to also test the CPU backends for completeness.

All modifications and new files are located within:

- `~/muDock/mudock/include/mudock/alpaka_implementation`: Containing all .hpp code files.
- `~/muDock/mudock/src/mudock/alpaka_implementation`: Containing all .cpp code files.

The specific files created and modified are:

- **Header files:** `adt_score_alpaka.hpp`, `alpaka_implementation.hpp`, `alpaka_random.hpp`, `alpaka_types.hpp`, `buffer_alpaka.hpp`, `genetic_alpaka.hpp`, `geom_transform_alpaka.hpp`, `invoke_kernel_alpaka.hpp`, `mutate_alpaka.hpp`, `queue_alpaka.hpp`.
- **Source files:** `adt_score_alpaka.cpp`, `genetic_alpaka.cpp`, `alpaka_random.cpp`, `geom_transform_alpaka.cpp`,

`queue_alpaka.cpp`.

The most critical implementations regarding MuDock’s kernels and the related optimizations are detailed below:

adt_score_alpaka:

This is the dominant kernel, accounting for **93% of the execution time** on the GPU, and it contains the majority of the applied optimizations. The primary modifications compared to the naive implementation are:

1. **alpaka::math:: for transcendental functions (FP32):** In the naive version, `::sin()`, `::cos()`, and `::sqrt()` from the standard `<cmath>` library were used. On the GPU, these functions operate in FP64, forcing an implicit `float→double→float` cast on every call. Replacing them with `alpaka::math::sin(acc, x)`, etc., maps directly to hardware FP32 instructions on CUDA (`__sinf`, `__cosf`).
2. **Warp-level Reduction** (`alpaka::warp::shfl_down`): The final score calculation requires a reduction (sum) of the contributions from all atoms for each chromosome. In the naive version, this reduction occurred in *shared memory* using a classic `barrier+loop` pattern. Substituting this with `alpaka::warp::shfl_down(acc, val, offset)` eliminates shared memory usage for intra-warp reductions by directly exploiting NVIDIA’s hardware registers (SHFL.DOWN PTX).
3. **Receptor Bounding Box in Constant Memory:** Originally, the bounding box coordinates of the receptor grid were fetched from Global Memory (HBM2e) upon every access. Because these parameters (`map_min`, `map_max`, `map_center`) are identical for all threads and strictly read-only, they were migrated to **Constant Memory** (`__constant__` in CUDA, mapped via `ALPAKA_STATIC_ACC_MEM_CONSTANT` in Alpaka). This allows the hardware to execute a *broadcast* access from the dedicated constant cache to all threads within a warp simultaneously, while keeping the multi-megabyte 3D potential grids in Global Memory.

geom_transform_alpaka:

This kernel applies geometric transformations. A fundamental aspect of this porting is the

structural mapping: **each ligand corresponds to a single Block** in the Alpaka grid, while **each atom within the ligand corresponds to a single Thread** within that block. Key design choices include:

- Using `ALPAKA_UNROLL` [8] for the atom loop on GPU targets to reduce branch overhead. On CPU backends, loop unrolling was explicitly disabled via `-DMUDOCK_DISABLE_UNROLL=ON` to prevent instruction bloat and L1 cache misses.
- Explicit indexing via `alpaka::getIdx<alpaka::Grid, alpaka::Blocks>(acc)[0u]` for the `ligand_id` and `alpaka::getIdx<alpaka::Block, alpaka::Threads>(acc)[0u]` for the `thread_id`. This aligns with the Alpaka design pattern, replacing CUDA’s `blockIdx.x` and `threadIdx.x`.

genetic_alpaka:

Implements the three evolutionary phases [9]:

- **initialize:** Generates the initial population of chromosomes with random positions and orientations.
- **iterate:** Performs a single GA generation, encompassing tournament selection, single-point crossover, and Gaussian mutation. It incorporates the on-device pseudo-random number generator (`alpaka_random.hpp`).
- **finalize:** Extracts the chromosome with the best overall score.

queue_alpaka:

Implements the bridge between MuDock’s TBB pipeline and the `alpaka::Queue`. Each worker operates its own private, non-blocking Alpaka queue. This design permits the asynchronous submission of GPU kernels from distinct CPU threads without requiring explicit locks.

3.2. Evaluated Backends

Because the testing environment was strictly limited to a single NVIDIA GPU, testing the application on Intel or AMD accelerators was not performed due to hardware availability constraints. While Alpaka’s design architecture theoretically ensures portability, executing empirical tests on diverse vendor hardware remains an important objective for future work. Regarding the CPU backends, only the serial and `ompCpu2Block()` accelerators were tested; this

Table 4: Evaluated backends and runtime flags.

Backend	Binary	-use flag	Notes
CPP Serial	build/cpp/muDock	CPP:CPU:0	Baseline reference, pure C++ without OMP
CPP OMP	build/cpp-omp/muDock	CPP:CPU:0[-7]	Inter-ligand parallelism with OpenMP
Alpaka Serial	build/alpaka-serial/muDock	ALPAKA:CPU:0[-7]	Alpaka backend on CPU, no intra-kernel OMP
Alpaka OMP	build/alpaka-omp/muDock	ALPAKA:CPU:0[-7]	Alpaka backend + intra-kernel OMP
CUDA Nativio	build/cuda/muDock	CUDA:GPU:0[:2:4GB]	GPU baseline, hand-written CUDA code
Alpaka CUDA	build/alpaka-cuda/muDock	ALPAKA:GPU:0[:2:4GB]	Porting subject of this work

choice was primarily motivated by the architectural constraints inherent to intra-block parallelism on CPUs.

The specific backends benchmarked and included in the final evaluation are listed below: Before delving into the results, it is important to specify how the implemented code was validated, along with the compilation and runtime flags used.

3.2.1 The Debugging Problem

Because molecular docking heavily relies on a Random Number Generator (RNG), establishing a consistent baseline of results is fundamentally difficult. This represented a critical challenge for strict validation. Fortunately, MuDock provides compilation flags that eliminate this non-deterministic component, allowing the establishment of a solid baseline to validate the Alpaka implementation. In debug mode (`-DMUDOCK_ENABLE_TEST=ON`), the `random_unit()` function always returns `fp_type{0.4f}`, making the application’s behavior entirely deterministic:

```
ALPAKA_FN_ACC ALPAKA_FN_INLINE fp_type
random_unit(alpaka_rand_state& state)
{
    if constexpr (is_debug()) {
        return fp_type{0.4f}; //
        deterministic mock
    } else {
        return static_cast<fp_type>(<
            next_random(state)) /
            static_cast<fp_type>(state.max
            ());
    }
}
```

To validate the code in debug mode, the following methodology was employed:

- **Mock RNG:** MuDock was compiled in Debug mode by enabling the CMake flag `-DMUDOCK_ENABLE_TEST=ON`. This forces the Pseudo-Random Number Generator (PRNG) to return a fixed value.
- **Fixed Seed:** During execution, the `-seed 42` argument was passed.
- **Result:** an exact bit-wise match was achieved between Native C++, Alpaka Serial, and Alpaka OMP on both the calculated energy and the genetic score. On the GPU, the discrepancy between Native CUDA and Alpaka CUDA was found to be **0.00004%**. This is a negligible delta deriving exclusively from the non-deterministic order of floating-point reduction at the hardware/warp level.

Compilation flags used for validation:

```
cmake -B build/alpaka-cuda \
  -DCMAKE_BUILD_TYPE=Debug \
  -DMUDOCK_ENABLE_TEST=ON \
  -DMUDOCK_BACKEND=ALPAKA_CUDA \
  -D__builtin_ia32_ldtilecfg=sizeof \
  -D__builtin_ia32_sttilecfg=sizeof ..
cmake --build build/alpaka-cuda -j$(nproc)
```

Execution command for single-core validation:

```
./build/alpaka-cuda/application/muDock \
  --protein data/1fkb/1fkb_pocket.pdbqt \
  --ligand data/small_1/small.adtmol2 \
  --use ALPAKA:GPU:0 \
  --search genetic \
  --population 20 --generations 10 --seed 42
```

Execution command for multi-worker validation (CPU ID clamping test):

```
./build/alpaka-serial/application/muDock \
  --protein data/1fkb/1fkb_pocket.pdbqt \
  --ligand data/small_1/small.adtmol2 \
```

```
--use      ALPAKA:CPU:0-7 \
--search genetic \
--population 20 --generations 10 --seed
42
```

This approach successfully validated the correct functioning of CPU ID clamping without the need to recompile the application, simply by altering the ID range within the `-use` parameter. (Note: *MuDock's* `-use` flag determines which backend is activated at runtime, following the grammar: `IMPLEMENTATION:DEVICE:IDS[:WORKERS[:MEMORY]]`)

Table 5: Validation results comparison across backends.

Backend	ADT Score (Evals)	Match vs CPP
CPP Serial	-16.119261	Baseline
Alpaka Serial	-16.119261	exact
Alpaka OMP	-16.119261	exact
CUDA Nativo	-16.119254	Delta 0.00004%
Alpaka CUDA	-16.119254	Delta 0.00004%

Note: The GPU delta is due to the non-deterministic order of floating-point sums during hardware reductions. It does not indicate a bug but is an intrinsic characteristic of SIMT floating-point operations [5].

4. Performance Benchmarking Setup

4.1. Experimental Setup

4.1.1 Execution Environment

All tests were performed on the **Accento** cluster, a machine provided by Politecnico di Milano. The environment was strictly isolated and managed via the **Spack** [4] package manager. The full hardware and software specifications of the compute node are reported in Table 6.

4.1.2 Datasets

All validation, debugging, and performance tests were executed using the Accento machine. Four distinct types of datasets were utilized to validate the project. For the CUDA backends, the following datasets were used (Table 7):

Table 7: Datasets used for GPU validation (ligand counts: Small 1.2M, Medium 1.2M, Big 842k). Molecule counts were verified via `grep -c "^@" <file.adtmol2>`.

Name	File	Size
Small	data/small_1/small.adtmol2	1.5 GB
Medium	data/mid_1/mid.adtmol2	2.0 GB
Big	data/big_1/big.adtmol2	2.0 GB

Note that these files are considered a legacy format by newer versions of MuDock. It was therefore necessary to implement backward compatibility directly within MuDock’s parser.

For the various CPU backends, a custom dataset was created instead (Table 8):

Table 8: Dataset used for CPU validation (500 ligands).

Name	File	Size
CPU Test	data/cpu_500lig.adtmol2	~9.4 MB

The **small/mid/big** datasets, containing hundreds of thousands of ligands, are designed to saturate the GPU. On the CPU, where the pipeline scales with the available physical cores (max 20 on this node), processing over a million ligands serially would require days; hence, this custom dataset was employed.

Table 6: Hardware and software specifications of the Accento compute node (palermo-fiorin-vm-1).

Component	Detail
Host CPU	AMD EPYC 7302 16-Core Processor — 2 NUMA nodes, 20 physical cores (1 thread/core, 2 sockets), Zen 2 architecture
Target GPU	NVIDIA A100-SXM4- 40GB
GPU Architecture	Ampere (sm_80)
GPU Memory	40 GB HBM2e, driver 550.78, CUDA 12.4
SM Count	108 Streaming Multiprocessors
Host RAM	N/A
OS	Ubuntu
Host Compiler	GCC 13.3.0
Device Compiler	NVCC 12.4.1
Spack	v0.21+

4.1.3 Build System and Profiling Suite

Build System: Compilation was managed using CMake [6], with separate targets created for each backend. The full production build commands, including all architecture-specific flags required on the Accento cluster, are reported below. Flags using `-DCMAKE_BUILD_TYPE=Debug` and `-DMUDOCK_ENABLE_TEST=ON` are specific to the validation and testing phase described in Section 3.

```
# CPP Serial
cmake -B build/cpp -DCMAKE_BUILD_TYPE=Release \
  -DMUDOCK_ENABLE_FAST=ON ..
cmake --build build/cpp -j$(nproc)

# CPP OpenMP
cmake -B build/cpp-omp -
  DCMAKE_BUILD_TYPE=Release \
  -DMUDOCK_ENABLE_FAST=ON \
  -DMUDOCK_ENABLE_OMP=ON ..
cmake --build build/cpp-omp -j$(nproc)

# Native CUDA
cmake -B build/cuda -DCMAKE_BUILD_TYPE=Release \
  -DMUDOCK_ENABLE_FAST=ON \
  -DMUDOCK_ENABLE_CUDA=ON \
  -DMUDOCK_ENABLE_OMP=ON \
  -DMUDOCK_GPU_ARCHITECTURES=cuda:sm_80 \
  -DCMAKE_CUDA_HOST_COMPILER=$CXX \
  -DCMAKE_CUDA_FLAGS="\
    -D__builtin_ia32_ldtilecfg=sizeof \
    -D__builtin_ia32_sttilecfg=sizeof"
cmake --build build/cuda -j$(nproc)

# Alpaka Serial
cmake -B build/alpaka-serial -
  DCMAKE_BUILD_TYPE=Release \
  -DMUDOCK_ENABLE_FAST=ON \
  -DMUDOCK_ENABLE_ALPAKA=ON \
  -DMUDOCK_ALPAKA_BACKEND=serial \
  -DMUDOCK_DISABLE_UNROLL=ON ..
cmake --build build/alpaka-serial -j$(
```

```
nproc)
```

```
# Alpaka OpenMP
cmake -B build/alpaka-omp -
  DCMAKE_BUILD_TYPE=Release \
  -DMUDOCK_ENABLE_FAST=ON \
  -DMUDOCK_ENABLE_ALPAKA=ON \
  -DMUDOCK_ALPAKA_BACKEND=omp2 \
  -DMUDOCK_DISABLE_UNROLL=ON ..
cmake --build build/alpaka-omp -j$(nproc)

# Alpaka CUDA
cmake -B build/alpaka-cuda -
  DCMAKE_BUILD_TYPE=Release \
  -DMUDOCK_ENABLE_FAST=ON \
  -DMUDOCK_ENABLE_ALPAKA=ON \
  -DMUDOCK_ALPAKA_BACKEND=cuda \
  -DMUDOCK_ENABLE_OMP=ON \
  -DMUDOCK_GPU_ARCHITECTURES=cuda:sm_80 \
  -DCMAKE_CUDA_HOST_COMPILER=$CXX \
  -DCMAKE_CUDA_FLAGS="\
    -D__builtin_ia32_ldtilecfg=sizeof \
    -D__builtin_ia32_sttilecfg=sizeof"
cmake --build build/alpaka-cuda -j$(nproc)
```

Relevant Compilation Flags:

- `-DMUDOCK_ENABLE_FAST=ON`: Injects `-O3 -ffast-math -march=native` for GCC and `-use_fast_math` for NVCC.
- `-DMUDOCK_GPU_ARCHITECTURES=cuda:sm_80`: Instructs `nvcc` to generate native SASS binaries for the NVIDIA Ampere (A100) architecture.
- `-DMUDOCK_DISABLE_UNROLL=ON`: Enabled *exclusively* on Alpaka CPU backends. It was observed during testing that Alpaka's deeply nested template inlining, when combined with aggressive loop unrolling, can cause instruction bloat on the CPU. This flag restores fairness in the benchmark by enabling the best-case architectural

scenario for Alpaka on CPU. On GPU, unrolling remains active.

- `-DMUDOCK_ENABLE_TEST=ON`: Enables the RNG mock to allow deterministic execution and validation.
- **GCC 13 / NVCC 12.4 Workaround:** The newer AMX headers in GCC 13 define two built-in functions (`__builtin_ia32_ldtilecfg` and `__builtin_ia32_sttilecfg`) that NVCC 12.4 fails to recognize, causing compilation errors when parsing standard library headers. The workaround involves stubbing these definitions directly via CMake:

```
target_compile_definitions(muDock PRIVATE
    __builtin_ia32_ldtilecfg=sizeof
    __builtin_ia32_sttilecfg=sizeof
)
```

Profiling Suite: To conduct a robust architectural analysis, data collection was partitioned between macroscopic and micro-architectural dimensions. For macroscopic evaluations and host resource tracking (such as Peak RSS, Context Switches, and global throughput), custom Python automation scripts were developed (`profiler_macro_global.py`, `profiler_macro_cpu.py` and `profiler_host_metrics.py`) interfacing directly with the Linux `/usr/bin/time -v` utility. At the micro-architectural level, CPU side metrics were collected using custom scripts developed for this project, whereas GPU timeline tracing and low-level hardware counter collection relied on NVIDIA Nsight Systems (`nsys`) and Nsight Compute (`ncu`), automated via the profiling infrastructure provided by the tutor in the `profile-scripts/gpu/nvidia/` directory.

Since this is an HPC project, validating the code in debug mode was not sufficient; the application must be highly efficient in production scenarios. Efficiency, in this context, implies the Alpaka implementation’s ability to match, or closely approach, the performance offered by MuDock’s pre-optimized native backends.

Comparisons were executed between Serial CPU and Alpaka Serial, CPU OMP and Alpaka OMP, and finally CUDA and Alpaka CUDA. However, it is important to note that the OpenMP comparisons are reported primarily for completeness and should not be interpreted as definitive upper bounds. This is because MuDock’s native

OMP implementation is currently incomplete, and Alpaka’s OMP backend presents specific design constraints based on the type of Acc utilized.

In summary, only after verifying correctness was the application recompiled to enable aggressive optimizations, allowing for an accurate evaluation of architectural throughput.

All backends were recompiled from scratch in `Release` mode using `-DMUDOCK_ENABLE_FAST=ON` (and `-use_fast_math` for NVCC on GPU targets). The complete per-backend compilation commands and flag rationale are detailed in Section 4.1.3. Datasets were selected as described in Section 4.1.2 (Tables 7 and 8), differentiating GPU and CPU workloads to correctly saturate available hardware resources.

4.2. Benchmarking Scripts

As described in Section 4.1.3, the benchmark execution was fully automated through a hybrid suite of Python scripts covering three dimensions: macroscopic throughput profiling (custom scripts, warm-up + 3 runs), host resource profiling (`/usr/bin/time -v`, Peak RSS and CPU utilization), and GPU micro-architectural profiling via Nsight Compute and Nsight Systems (limited to 20 iterations due to Kernel Replay overhead). The complete tool descriptions and attribution are provided in Section 4.1.3.

5. Results

The most significant results obtained during the benchmarking phase are presented in this section. The evaluation was systematically divided into two main areas: CPU-bound execution, aimed at analyzing single-thread efficiency and OpenMP scalability, and GPU-bound execution, focused on massive parallel throughput and hardware saturation. To maintain the conciseness of this report, only the most representative charts have been included. The analysis begins with a detailed look at the CPU performance.

5.1. CPU Serial and Multi-Worker

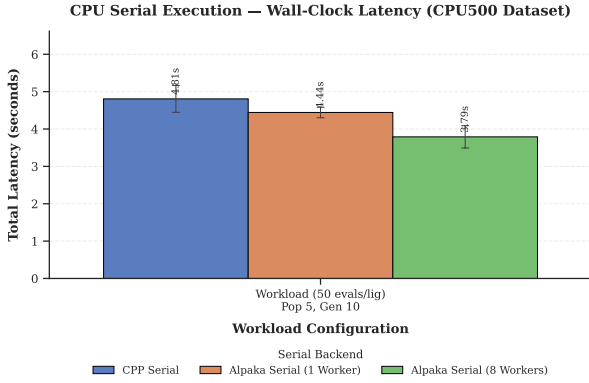


Figure 1: Serial CPU Latency on the cpu500 dataset.

As evidenced by the consolidated benchmark charts on the dedicated dataset (cpu_500lig, Pop: 5, Gen: 10), the throughput of the native **CPP Serial** backend settles at approximately **10.44 Evals/s**, corresponding to a latency of 4.81 seconds. In contrast, the **Alpaka Serial (1 Worker)** implementation achieves **11.26 Evals/s**, reducing the latency to 4.44 seconds.

This outcome is particularly significant. Alpaka’s template metaprogramming is designed to produce zero-overhead abstractions. This result is consistent with that design goal, empirically confirming that no runtime overhead is introduced during baseline serial execution. The slight measurable speedup in Alpaka is largely attributable to the modern GCC 13 compiler, which applies highly aggressive inlining to Alpaka’s deeply nested template structures, generating machine code that is marginally better optimized than the hand-written native C++

counterpart.

Table 9: Complete CPU throughput on the cpu500 dataset (Pop: 5, Gen: 10).

Backend	Work.	Lat (s)	Evals/s
CPP Serial	1	4.81	10.44
Alpaka Serial	1	4.44	11.26
CPP OMP	1	2.14	23.40
Alpaka OMP	1	5.57	8.98
CPP OMP	8	1.56	32.52
Alpaka OMP	8	1.45	34.51

*Note on Table 9: When operating with a single worker, the native **CPP OMP** backend achieves significantly higher throughput than **Alpaka OMP**. This disparity arises because the native OpenMP implementation relies heavily on SIMD vectorization rather than true multithreading, granting it an advantage in single-worker serial execution compared to a single Alpaka thread.*

5.2. CPU OpenMP

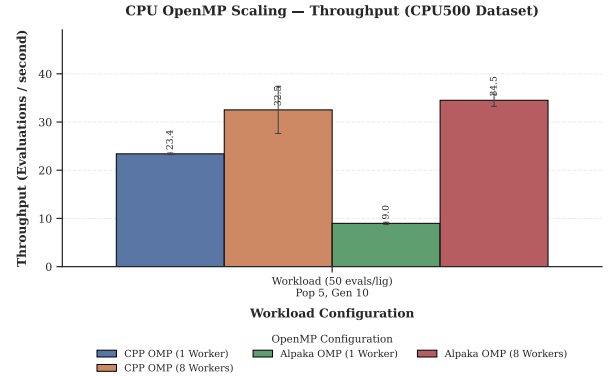


Figure 2: CPU OpenMP Throughput.

Moving from single-worker to multi-worker scenarios. By increasing the pipeline workers to 8, the native **CPP OMP (8 Workers)** version scales appropriately, reaching a throughput of **32.52 Evals/s**. Nevertheless, it is once again surpassed by the **Alpaka OMP (8 Workers)** backend, with a performance landscape by achieving **34.51 Evals/s**.

Disclaimer regarding the OpenMP comparison: It is important to clarify that the direct throughput comparison between the native **CPP OMP** and **Alpaka OMP** backends is presented here purely for demonstrative purposes. Although the results are reported following the

same topology and metrics as the other backends for the sake of consistency, MuDock’s native OMP backend is actually a legacy component. It does not fully implement standard OpenMP multithreading, relying instead on basic SIMD vectorization. Therefore, this comparison serves exclusively to illustrate how Alpaka behaves when targeting a different backend and to highlight specific architectural limitations encountered with Alpaka’s OpenMP mapping.

5.3. Indicative Metrics: Host Resources (To be Discussed)

The analysis of host resources highlights how the different backends manage memory and processor scheduling. The following metrics were collected:

1. **Host Footprint on CPU:** Alpaka CPU backends consume ~ 196 MB of Peak RSS compared to ~ 143 MB for native C++. This 50 MB increase is expected, as Alpaka instantiates internal host-side structures (queues, grid emulations) not required by native code.
2. **Abstraction Efficiency on GPU:** Conversely, GPU memory footprints are virtually identical (native CUDA: ~ 999 MB, Alpaka CUDA: ~ 978 MB). The baseline increase is strictly tied to the heavy CUDA runtime initialization, confirming that Alpaka introduces zero additional host-side overhead.
3. **CPU Utilization:** When scaling to 8 Workers, Alpaka OMP saturates cores more aggressively, reaching up to **1055% CPU utilization** (measured via `/usr/bin/time -v`) compared to 815% for CPP OMP. While this correlates with slightly higher throughput, it is artificially inflated by architectural constraints rather than efficient parallel execution.
4. **Context Switches:** Figure 5 highlights a pronounced anomaly in the OpenMP execution. While native CPP OMP maintains $\sim 161,900$ context switches, Alpaka OMP triggers over **1,193,000**. When multiple asynchronous workers concurrently trigger parallel regions across the shared physical cores, frequent thread preemption and OS scheduling events occur, underscoring the need for deeper thread-pinning optimiza-

tions in future work.

Note: These host-level profiling metrics reflect qualitative runtime trends under concurrent workloads. The complex interaction between asynchronous TBB task and OpenMP thread pools highlights the need for dedicated thread-affinity and core-pinning investigations in future work.

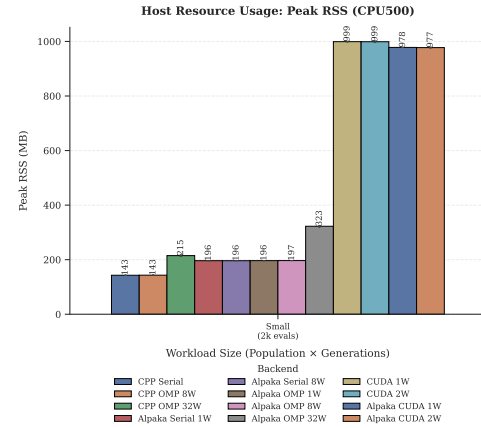


Figure 3: Peak Resident Set Size (RSS).

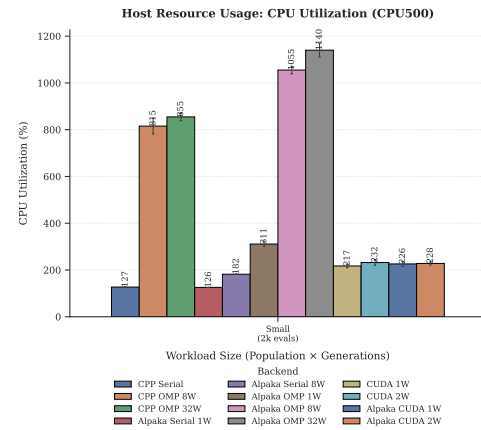


Figure 4: Host CPU Utilization.

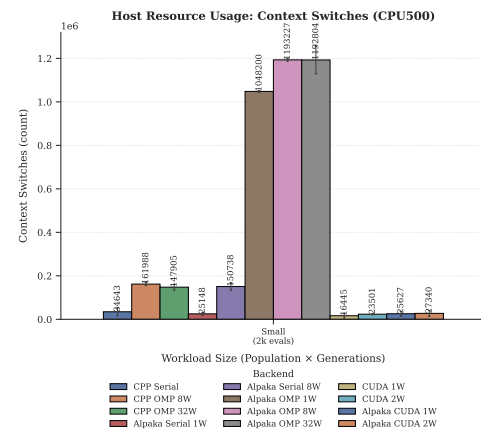


Figure 5: Host Context Switches.

5.4. GPU Performance and Scaling

This section evaluates the execution throughput (Evaluations per second) of the Alpaka porting compared to the native CUDA implementation on the NVIDIA A100.

To provide a comprehensive overview, the benchmarks were divided into two distinct configurations reflecting different computational burdens per molecule:

1. **Low-Medium Burden (Small/Mid Datasets):** Configured with a lighter genetic exploration space (e.g., Pop: 20, Gen: 100 for Small, yielding 2,000 evaluations per ligand).
2. **High Burden (Big Dataset):** Designed as a severe stress test, configured with a massive genetic exploration space (Pop: 100, Gen: 400, yielding 40,000 evaluations per ligand). This drastically increases the arithmetic intensity per molecule, resulting in a naturally lower absolute throughput compared to the smaller datasets.

5.4.1 Scaling on Low-Medium Burden Workloads

Table 10 reports the execution throughput on the Small and Mid datasets. On lightweight workloads (Small dataset), native CUDA maintains a modest throughput advantage of approximately 5–6% across both 1 and 2 workers (640 vs 608 Evals/s and 898 vs 845 Evals/s). However, as the computational density per ligand increases (Mid dataset), this overhead amortizes rapidly. Single-worker execution yields virtually identical performance (41.06 vs 41.01 Evals/s), while scaling to 2 workers demonstrates comparable scaling efficiency for both backends (284.5 Evals/s for Alpaka vs 274.4 Evals/s for CUDA). Overall, the two implementations demonstrate comparable scaling behavior across low and medium workloads.

Table 10: GPU Throughput across Small and Mid Datasets.

Dataset	Work.	CUDA (Ev/s)	Alpaka (Ev/s)
Small	1	640	608
Small	2	898	845
Mid	1	41.06	41.01
Mid	2	274.4	284.5

5.4.2 Scaling on High Burden Workloads

Figure 6 illustrates the application behavior under the massive computational load of the Big dataset. In a single-worker configuration, the Alpaka implementation achieves 18.29 Evals/s. The negligible 0.3% difference compared to native CUDA (18.35 Evals/s) falls entirely within the expected statistical variance of PCIe data transfers.

When configured with 2 concurrent workers, both backends successfully mask host-to-device latency by overlapping DMA memory transfers with kernel execution, fully saturating the Streaming Multiprocessors. The throughput scales symmetrically, reaching 114.62 Evals/s for Alpaka and 114.47 Evals/s for CUDA.



Figure 6: GPU Throughput on the Big dataset (Pop: 100, Gen: 400).

Note on validation methodology: The deliberate variation of datasets, populations, and generations across tests was designed to evaluate Alpaka under diverse stress conditions. The core assumption of this project is relational: if native CUDA is known to be highly optimized on this hardware, proving that Alpaka precisely mirrors CUDA’s behavior inherently validates Alpaka’s efficiency. Therefore, the absolute throughput values are less significant than the comparative behavioral mapping. Ultimately, the results are virtually identical across all scenarios, with the only minor distinction being a slightly higher execution variance observed in Alpaka during testing.

5.5. Micro-Architectural Analysis

To analyze the hardware scheduling and instruction-level execution of the generated code, a profiling campaign was conducted using NVIDIA Nsight Systems (`nsys`) and Nsight Compute (`ncu`).

As previously mentioned, the collection of these advanced metrics was automated utilizing the Python infrastructure provided by the instructors (`profile-scripts/gpu/nvidia/`). To isolate a single genetic algorithm iteration, the specific kernels were targeted directly using the profiler’s regex filtering. To mitigate the measurement overhead introduced by kernel replay during `ncu` profiling, the provided scripts limited executions to 20 iterations. Additionally, the NVCC compiler reports were extracted (`ptxas`) to evaluate static register allocation.

Disclaimer regarding absolute performance: It is crucial to note that the absolute micro-architectural metrics presented below deviate from initial expectations. In the generated Roofline plots, the kernels operate in the lower region of the chart (indicating low performance and bandwidth utilization), when MuDock’s native CUDA backends typically operate in the upper-right region under real-world conditions. However, the operational points of native CUDA and Alpaka CUDA are perfectly superimposed. This strongly supports the hypothesis that the observed performance ceiling is an artifact of the specific datasets and hardware configuration utilized for these tests, rather than a flaw in the kernel implementations. Consequently, as stated previously, these metrics must be interpreted purely comparatively. For an exhaustive performance analysis of MuDock, the reader is referred to the original publications by the development team; the primary objective of this profiling campaign is strictly to evaluate the Alpaka abstraction layer, not MuDock’s absolute capabilities.

5.5.1 `adt_score` Kernel

The `adt_score` kernel evaluates the molecular binding energy.

Profiling via Nsight Systems reports near-identical execution times: 10.47 ms for native CUDA and 10.52 ms for Alpaka. Furthermore, the `ptxas` compiler logs confirm that both implementations demand exactly 64 registers per thread.

Looking at the Single-Precision Roofline models (Figures 7 and 8), the two backends perform

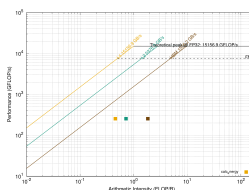


Figure 7: CUDA `adt_score` Roofline.

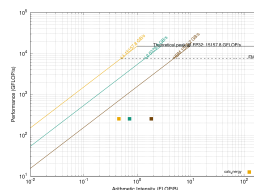


Figure 8: Alpaka `adt_score` Roofline.

practically on par, achieving 254.40 GFLOPS and 252.60 GFLOPS respectively. Crucially, the arithmetic intensity matches perfectly across the entire memory hierarchy (L1, L2, and HBM). The achieved occupancy for both is firmly bottlenecked at 6.24%, a direct consequence of the 64-register limit mentioned above.

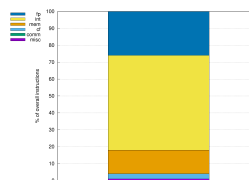


Figure 9: CUDA `adt_score` InstMix.

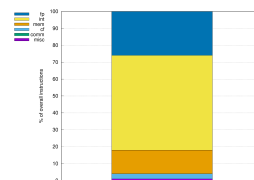


Figure 10: Alpaka `adt_score` InstMix.

Finally, the Instruction Mix distribution (Figures 9 and 10) is perfectly equivalent: 56.0% integer operations, 26.0% floating-point, 14.0% memory accesses, and 3.0% control flow. This effectively proves that the compiler generate machine code for Alpaka that is structurally indistinguishable from the hand-written CUDA version.

5.5.2 `geom_transform` Kernel

The `geom_transform` kernel handles the spatial transformations of the molecular structures.

Interestingly, profiling via Nsight Systems reveals that Alpaka slightly outperforms native CUDA on this specific task, recording an execution time of 673 μ s compared to 694 μ s. The root cause of this improvement is evident in the `ptxas` compiler logs: the Alpaka implementation requires only 48 registers per thread, a significant reduction compared to the 72 registers demanded by the native CUDA version. This reduction in register pressure is rooted in Alpaka’s internal argument manage-

ment. While the native CUDA `__global__` kernel takes over 20 separate raw pointers (increasing parameter-passing register allocation overhead), Alpaka’s `createTaskKernel` API internally packs all kernel arguments into a single aggregate `std::tuple` structure before device dispatch. This C++ encapsulation allows the `ptxas` compiler to optimize parameter loading more efficiently, inherently lowering the initial register footprint of the kernel.

This alleviated register pressure directly benefits the hardware scheduler.

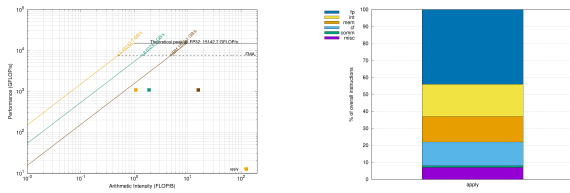


Figure 11: Alpaka `geom_transform` Roofline.

Figure 12: Alpaka `geom_transform` Inst-Mix.

Methodological Note on Profiling Artifacts: Roofline and Instruction Mix plots for the native CUDA `geom_transform` kernel are omitted from this section. Although Nsight Compute successfully intercepted the kernel and completed the replay cycles, the post-processing script failed during metric extraction, returning NaN values. This profiling artifact is likely due to the kernel’s intensive use of transcendent functions (sine/cosine) executed on Special Function Units (SFUs), which often evade standard FMA/ALU counters expected by the script.

5.5.3 `iterate` Kernel

The `iterate` kernel manages the core evolutionary phases of the genetic algorithm, encompassing selection, crossover, and mutation. Profiling via Nsight Systems reveals a notable performance advantage for Alpaka, which records an execution time of 44.8 μ s compared to 56.6 μ s for native CUDA. The advanced metrics extracted via Nsight Compute pinpoint the source of this speedup: an improvement in the Uniform Branch metric, which reaches 76.01% for Alpaka versus 62.45% for native CUDA. A higher Uniform Branch percentage indicates reduced warp divergence, meaning that threads within the same warp are more consistently tak-

ing identical execution paths. (Note: This substantial improvement in branch uniformity was not anticipated and warrants further investigation to determine whether it is a systemic benefit of Alpaka’s code generation or a measurement artifact).

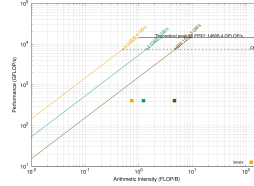


Figure 13: CUDA `iterate` Roofline.

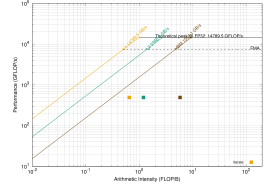


Figure 14: Alpaka `iterate` Roofline.

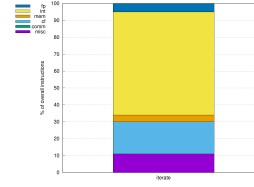


Figure 15: CUDA `iterate` InstMix.

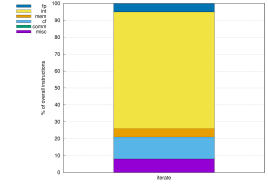


Figure 16: Alpaka `iterate` InstMix.

The corresponding Roofline and Instruction Mix models (Figures 13 to 16) further corroborate this efficient mapping, demonstrating that the structural integrity of the execution is fully preserved by the compiler despite Alpaka’s high-level abstraction. *Note on Kernel Selection: The micro-architectural analysis presented above focuses exclusively on the `adt_score`, `geom_transform`, and `iterate` kernels. Together, these represent the absolute core of the computational workload. Minor setup and memory-management routines were intentionally excluded from the deep profiling campaign, as their negligible runtime impact would not yield significant comparative insights.*

5.6. Aggregate Application Roofline Model

Figure 17 presents the aggregated Application Roofline Model, which captures the macro-level execution of the entire MuDock application on the Big dataset (2 Workers).

As anticipated in the methodological disclaimer, this plot is intended purely for a qualitative behavioral comparison. The application is far from

saturation of the hardware, utilizing only 16.7% of the available memory bandwidth and around 10.9% of the Streaming Multiprocessor (SM) computational throughput.

Ultimately, Alpaka mirrors the native CUDA behavior closely.

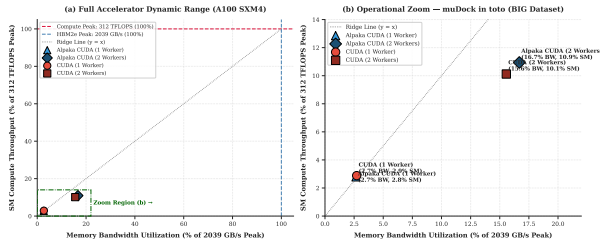


Figure 17: Application-level Roofline Model (Big dataset, 2 Workers).

6. Results Evaluation and Future Works

6.1. Results Assessment

With the quantitative metrics established, this section provides a concise assessment of the overall results.

Overall, the results are coherent and aligned with the initial expectations. On the GPU side, while there is always room for yet another micro-optimization, a point of "parity" has effectively been reached. The initial research question is fully answered:

- **Zero Performance Penalty (Static Polymorphism):** Alpaka faithfully mirrors the execution efficiency of native CUDA on the A100. The ability of Alpaka to match native CUDA throughput without runtime penalties is formally guaranteed by its architecture. As documented in the original Alpaka publication [8], the framework strictly avoids dynamic polymorphism (virtual functions) in the execution path. Instead, it relies on trait-based static polymorphism and aggressive template inlining. This guarantees that the abstraction is entirely resolved at compile-time, resulting in a zero-overhead abstraction where the generated PTX machine code is structurally identical to hand-written CUDA.
- **Abstraction Verification:** The compiler entirely strips away Alpaka's C++ templates, leaving an instruction footprint indistinguishable from hand-optimized code.
- **Functional vs. Performance Portability:** A unified C++ codebase now runs across distinct architectures. However, while performance parity was successfully validated on the primary GPU target and serial CPU execution, achieving and guaranteeing comparable architectural efficiency across all parallel CPU backends (such as OpenMP) and additional hardware accelerators remains an ongoing effort reserved for future work.

6.2. Project Considerations

For the more curious readers, this section shifts the focus to the actual engineering journey, presenting the problems, solutions, and insights gathered along the way.

The **What was accomplished** subsection outlines the true starting point of the project and explains, as concisely as possible, the concrete development steps undertaken. Following that, **What was not accomplished (Future Work)** explores what was intentionally left out of the scope, detailing the rationale behind those design choices. Finally, the **Conclusions** subsection presents the ultimate takeaways not just on the final product, but on the limitations and challenges encountered throughout the process bringing this report to its definitive close.

6.2.1 What was accomplished

Before discussing the final achievements, it is important to understand the starting point of this work. The MuDock repository contained an `alpaka-dev` branch featuring a very rough, naive draft of the Alpaka port. The main issue was that its structure was completely misaligned with the native CUDA backend, and the code lacked any real optimization.

The first objective was to establish a file and directory structure for the Alpaka backend that was completely symmetrical to the CUDA one. This was crucial to ensure that comparing the two implementations would be straightforward and coherent. While perfect code-level symmetry is impossible given that native CUDA relies on lower-level, hand-optimized paradigms, a functionally identical structure was successfully created.

During this alignment process, some inherent architectural differences became apparent. Specifically, the Alpaka backend lacks certain files that are heavily utilized in CUDA:

- **Absence of `cuda_utils.cuh` (RAII and Modern C++):** In the native backend, `cuda_utils.cuh` provides C-style macros (e.g., `MUDOCK_CHECK`) necessary to catch and interpret `cudaError_t` codes returned by API calls. Alpaka eliminates this requirement entirely by leveraging modern C++ paradigms. It uses RAII (Resource Acquisition Is Initialization) for resource allocation and internally manages failures by throwing standard C++ exceptions.

In short, Alpaka's higher level of abstraction removes the need for hardware-specific boilerplate. However, as a natural trade-off, this abstraction necessitated the in-

troduction of new, framework-specific files such as `buffer_alpaka.hpp` and `invoke_kernel_alpaka.hpp`. While CUDA relies on native compiler syntax such as raw pointers and the `<<...>>` chevron operator to launch kernels, Alpaka requires formal C++ objects and explicit API calls to map these identical actions.

While transforming that initial naive draft into a cohesive and symmetrical implementation, a preliminary test was executed to verify the baseline. This immediately brought to light the first interesting technical anomaly of the project, even before any actual kernel optimizations could begin.

6.2.2 The Random Number Generation Anomaly

How to generate random numbers correctly? This is the first interesting point of the project that warrants discussion.

Initially, the naive Alpaka implementation completely lacked a dedicated file for random number generation. After introducing `alpaka_random.hpp` and `alpaka_random.cpp` to make the code structurally similar to CUDA, without overcomplicating things, the tools provided by the library were investigated, and `alpaka::rand::Philox4x32x10` was chosen, an ideal generator for GPU systems.

Once this first structural alignment was done, the first comparative test was executed. The results were surprising: Alpaka was extremely more performant than CUDA, despite lacking any kind of optimization.

Investigating the root of this anomaly, it turned out to be more trivial than expected. In the CUDA file, the original default NVIDIA generator (`curandState`, typically based on XORWOW) was removed and replaced with `curandStatePhilox4_32_10_t`. (It should be noted that there was a `TODO` comment above this generator hinting at the possibility that it was not very efficient).

Once this small modification was made (and having unintentionally optimized CUDA as well in the process) the results returned to being coherent, with Alpaka falling well below CUDA due to all its missing optimizations.

Note: One could open a separate report on why

simply changing the random number generator changes performances so much. References to the actual generator are attached here to avoid excessively lengthening an already long report [15].

Finally, before moving forward with the optimizations, it is important to mention the fact that now:

6.2.3 Dynamic Generator Selection

In the `alpaka_random.hpp` file, the implementation was not limited to creating a single random class. Instead, C++ metaprogramming was used (via `std::enable_if_t`) to perform a compile-time dispatch of the pseudo-random generator based on the target hardware:

- If the chosen Alpaka backend is a CPU (`alpaka::PlatformCpu`), the code automatically instantiates a Mersenne Twister (`alpaka::rand::engine::cpu::MersenneTwister`), which is the RNG of choice and extremely performant on x86 architectures. [7].
- If instead the accelerator is a GPU, the code falls back on the default, namely Philox 4x32x10 (`alpaka::rand::Philox4x32x10`), a counter-based generator perfect for SIMT because it has a very small memory state.

Thus, the selection of the generator is much cleaner and more efficient based on the chosen backend, so as not to have to mention it later. This was done because only later in the project was it decided to also look at the CPU implementation.

Having addressed this first point, it is time to move on to the problems encountered in managing Alpaka and optimizing the kernels. Note that the major optimizations and details have already been reported in the previous sections, so to avoid repetition, the focus will move directly to some considerations:

6.2.4 1. Alpaka is not an easy library.

Due to its nature and syntax, sometimes understanding what was actually happening under the hood was truly complicated. This complexity is largely related to memory management and, above all, with the OMP backends. Additionally, instructions (like `warp_shuffle`, for example) exhibited peculiar behaviors based on the

active backend (such as degenerating into a no-op identity return on single-threaded CPU backends). These problems bring us to the second consideration.

6.2.5 2. The different paradigms of memory management and parallelization.

- **Global Memory:** CUDA allocates VRAM via `cudaMalloc` and transfers data using `cudaMemcpy`. This model relies directly on raw pointer manipulation, leaving memory safety entirely to manual tracking and exposing the application to segmentation faults if byte allocations are miscalculated. In contrast, Alpaka completely abstracts raw pointers in global memory through strongly-typed `Buffer` objects (`alpaka::allocBuf`) that natively track dimensionality (1D, 2D, 3D). Host-to-device transfers are executed via `alpaka::memcpy`, which operates directly on typed buffer objects rather than raw byte counts, guaranteeing type-safety at compile-time.
- **Shared Memory (Intra-block):** In CUDA, to share data between threads of the same block, it is enough to use the keyword `__shared__ float my_array[256];` directly inside the kernel. Alpaka, since it must also run on CPUs (where physical shared memory does not exist and L1/L2 cache is used), formalizes the allocation. Inside the kernel, special keywords cannot be used. The developer must explicitly ask the accelerator to instantiate the memory via a dedicated API (e.g., `alpaka::declareSharedVar`). The accelerator object takes care of allocating that variable in the true Shared Memory on a GPU, or as a simple local array (thread-safe) if the backend is a CPU.
- **Constant Memory:** In CUDA, an ultra-fast cached memory is available, accessed by declaring global variables with `__constant__`. In Alpaka, although there is no modern object-oriented API like `allocBuf` for constant memory, there is a dedicated macro workaround: `ALPAKA_STATIC_ACC_MEM_CONSTANT` combined with `alpaka::DevGlobal`. In the `adt_score_alpaka` implementation, this

API was successfully utilized to store the receptor grid bounding box parameters (`map_min_const`, `map_max_const`, `map_center_const`), ensuring they are cached efficiently across all threads. However, this approach exposes a 'leaky abstraction' in Alpaka, as it forces the developer to declare C-style static global variables outside the kernel scope, breaking the otherwise pure object-oriented Memory Hierarchy model of the framework.

- **Zero-Copy and Mapped Memory:** In CUDA, zero-copy pinned host memory is traditionally allocated via `cudaHostAlloc` (with the mapped flag) and accessed on the device through raw pointers. Alpaka formalizes this concept via `alpaka::MappedBuf`. It allows explicitly declaring a buffer physically allocated in host RAM but mapped directly into the accelerator's address space. The abstraction is substantially cleaner and safer because compile-time Memory Traits prevent passing an incorrectly mapped buffer to an incompatible device.

6.2.6 3. The lack of actual directives.

Finally, the third consideration relates to the absence of certain native directives. However, these are not missing features that will be introduced later, but rather design choices consistent with the true Alpaka paradigm. For example:

- **No Shuffle on CPU:** OpenMP threads are completely independent OS threads. There is no hardware instruction on a CPU to exchange registers "on the fly" like warps do on a GPU.
- **Lack of Library Primitives (e.g., NVIDIA CUB):** Alpaka is a core abstraction framework, meaning it lacks a higher-level library of collective operations equivalent to NVIDIA CUB. Standard primitives like Prefix Scan, Radix Sort, or explicit Block/Grid Reductions are not natively provided. The developer is forced to reinvent the wheel, manually implementing these primitives using shared memory or warp shuffles, which noticeably increases development time (as experienced when rewriting the reduction in the `adt_score` kernel).

- **Blocking Barriers:** While on a GPU `alpaka::syncBlockThreads(acc)` is an almost free operation (costing just a few clock cycles), on the `AccCpuOmp2Threads` backend it translates into a software OpenMP barrier (`#pragma omp barrier`). This generates a big overhead (context switches, spin-locks) every time a synchronization is attempted.

6.2.7 4. Compile-Time Constraints and Unrolling

In pure CUDA, the block size can be passed seamlessly as a template parameter (`template <int BLOCK_SIZE>`), allowing the compiler to perfectly unroll loops via `#pragma unroll`. In Alpaka, due to its agnostic abstraction, the dimensions of the Grid and Block are generally extracted at runtime via `alpaka::getWorkDiv<...>`. However, compiler unroll directives demand strict compile-time constants. This conflict "breaks" dynamic unrolling, forcing the injection of C-style static macros (e.g., `MUDOCK_ALPAKA_BLOCK_SIZE`) to force the unroll. While functionally correct, this requirement pollutes the elegance of the modern C++ codebase.

6.2.8 5. Absence of Hardware-Specific Units (e.g., Texture Memory)

During the initial porting phase, the presence of a `cuda_texture.cuh` file in the native codebase prompted an investigation into how Alpaka handles hardware texture mapping. This search revealed a fundamental limitation of the Alpaka ecosystem: the complete absence of a Texture Memory API.

This omission is a direct consequence of Alpaka's "Performance Portability" philosophy. Since the framework must compile and run seamlessly on standard x86 CPUs—which inherently lack dedicated texture mapping units—it deliberately refuses to expose abstractions for vendor-specific hardware features. Consequently, any GPU algorithm heavily reliant on hardware-accelerated interpolation, Surface Objects, or proprietary Texture Caches cannot be ported to Alpaka natively.

Although further inspection revealed that the native CUDA implementation of MuDock actu-

ally opted for software-based trilinear interpolation via Global Memory (making this limitation practically irrelevant for this specific porting effort), it remains a critical structural constraint. Developers migrating texture-bound codebases are forced to rewrite logic blocks via software using standard memory buffers, thereby losing access to the underlying specialized silicon and potentially incurring severe performance penalties.

Despite these initial considerations and challenges, it must be specified that all fundamental optimizations were successfully applied. Therefore, it is important to remind the reader that the final results reported identical behaviors. (Note, however, that when scaling across multiple workers, Alpaka occasionally exhibited slight performance degradation compared to CUDA under specific runs, which aligns with the higher execution variance previously mentioned in the Results section).

Final note on the implementation: For the sake of brevity, the numerous minor micro-optimizations, structural refactoring, and extensive debugging steps undertaken to achieve this functional parity have not been detailed in this report. The complete history of the implementation, including all commits and architectural tests, is publicly available and documented in <https://github.com/Oleg-Nedina/muDockON>.

6.2.9 What was not accomplished (Future Work)

It is important to remind the reader that the core objective of the project itself was GPU portability, and only subsequently was introduced a CPU analysis (which, in serial execution, exhibited the exact same behavior). So, leaving aside the micro-optimizations that are lost in favor of portability, what was truly not done? OMP. And by OMP, this refers to a true validation and optimization for the OMP backend.

As mentioned earlier, MuDock does not present a mature native OMP backend for comparison, and therefore it was fundamentally impossible to validate `alpaka_omp`. It is important, however, to mention the issues that this backend presented during the testing phase. Alpaka has two diversified OMP backends:

6.2.10 1. `alpaka::AccCpuOmp2Blocks` (Inter-Block Parallelism)

This is the default accelerator and the most efficient for traditional CPUs.

- **How it works:** Alpaka maps the OS OpenMP threads to the Grid Blocks. Every OpenMP thread (and therefore every physical core of the CPU) takes charge of an entire Block. The "Threads" inside that block are not executed in parallel, but in a purely sequential manner via a classic `for` loop generated by the compiler.
- **Why it is fast:** Since the threads in the block are executed sequentially by the same core, the host C++ compiler (like GCC) can perform automatic vectorization (SIMD/AVX) of the loop. Furthermore, there is no synchronization overhead: the `alpaka::syncBlockThreads(acc)` instruction is literally ignored (it becomes a no-op), since there are no true concurrent threads inside the block that need to be synchronized.

6.2.11 2. `alpaka::AccCpuOmp2Threads` (Intra-Block Parallelism)

This is the accelerator that was tested to attempt forcing parallelization on the single atoms inside a single ligand.

- **How it works:** Alpaka inverts the mapping. It maps the OpenMP threads directly to the Threads inside the block. If `MUDOCK_ALPAKA_BLOCK_SIZE` is configured to 32, Alpaka generates a `#pragma omp parallel for` that forces the CPU to "wake up" 32 physical/logical OpenMP threads to execute the 32 block threads in true concurrency. Generating 32 OS threads to process extremely short cycles creates a lot of overhead in context switching and thread management. Worse still, whenever `alpaka::syncBlockThreads(acc)` is invoked, Alpaka is forced to insert a blocking `#pragma omp barrier`, forcing the CPU cores to wait for each other and killing the overall throughput.

It should be noted that the use of `AccCpuOmp2Threads` actually required modifying how the kernel code was written just to

pass basic mathematical validation. Simply swapping the accelerator type led to immediate problems during energy calculations (the output values diverged significantly from the baseline), and modifying the code to fix this correctness issue led directly to the aforementioned performance degradation. Furthermore, regarding `AccCpuOmp2Blocks`, Alpaka’s internal implementation features a comment explicitly stating that it requires the block size parameter to be set to 1. An attempt was made to force a different value anyway, which immediately resulted in a runtime error (as previously anticipated).

6.2.12 Investigating OpenMP Resource Utilization

As mentioned earlier, `AccCpuOmp2Threads` presented structural limitations inherent precisely to MuDock’s architecture, so the focus shifted to analyzing the behavior of `AccCpuOmp2Blocks`. During a discussion with Davide Gadioli, the CPU resource utilization of this accelerator was monitored to verify whether the underlying OpenMP parallelization was actually functioning correctly. The natural expectation was that launching the program would automatically trigger OMP parallelization, exploiting all available hardware resources.

The problem emerged when observing the CPU utilization via standard Linux diagnostic tools like `top` or `htop`. The processor utilization consistently hovered strictly around $100\% \times (\text{number of TBB workers launched})$. For instance, launching the application with 8 workers resulted in roughly 800% CPU utilization, completely ignoring the remaining available cores (even if `OMP_NUM_THREADS` was set to a higher value). It appeared as though the OpenMP backend was not parallelizing as expected.

The most solid hypothesis to explain this behavior ties back to how this specific backend functions. Since MuDock processes ligands individually, the Alpaka Grid for the kernel is instantiated with exactly 1 block. Because `AccCpuOmp2Blocks` delegates OpenMP parallelization exclusively at the block level, the generated `#pragma omp parallel for` construct is presented with only a single outer iteration to process. Consequently, each worker process occupies exactly one core (100% utilization) to

process its single block, rendering any additional OpenMP thread allocation entirely useless for intra-ligand parallelization.

6.2.13 The Loss of Vendor-Specific Conveniences: Manual Heuristics

To conclude this section, it is worth discussing a broader category of features that were ultimately left behind: micro-optimizations tied to vendor-specific APIs. In general, almost the entire execution pipeline was successfully ported to the GPU, except for those highly specialized functions that inherently lack a universal directive in an agnostic framework.

A perfect example is the calculation of theoretical occupancy to define the ideal batch size of ligands. Native CUDA utilizes an internal Occupancy Calculator API, which predicts exactly how many blocks can fit on a Streaming Multiprocessor based on dynamic register usage, automatically saturating the hardware. In Alpaka, such a universal function does not and cannot exist. Because the library compiles the identical C++ codebase for x86 CPUs, AMD GPUs, and NVIDIA hardware, it cannot provide an API based on register pressure, since registers behave and are managed entirely differently across these architectures.

Deprived of this low-level convenience, it was necessary to implement a manual, compile-time heuristic to emulate this behavior, injecting explicit hardware knowledge directly into the code. Knowing that as the number of atoms (`n_atoms`) increases, the register pressure rises and consequently lowers the occupancy, a static decay of blocks per SM was mapped using `if constexpr`:

```

if constexpr (MUDOCK_ALPAKA_BLOCK_SIZE
    == 1) {
    // CPU backends
    bucket_multiple = {1, num_sms};
} else {
    // GPU backends
    // Heuristic simulation of
    // occupancy (register pressure
    // vs n_atoms)
    int blocks_per_sm = 16;
    if constexpr (n_atoms > 64)
        blocks_per_sm = 12;
    if constexpr (n_atoms > 128)
        blocks_per_sm = 8;
    if constexpr (n_atoms > 192)
        blocks_per_sm = 4;
    bucket_multiple = {blocks_per_sm,
        num_sms};
}

```

This workaround demonstrates a trade-off: when utilizing cross-platform abstractions like Alpaka, the developer loses access to powerful vendor-specific tools, often forcing them to rely on architectural "reverse engineering" and manual heuristics to maintain peak efficiency.

6.2.14 Future Work Summary

Building upon the limitations and anomalies discussed above, the immediate directions for future work on the MuDock Alpaka port can be summarized as follows:

- **OpenMP Investigation:** A deeper, dedicated investigation into the OMP backends to definitively resolve the intra-ligand scaling bottlenecks.
- **Heuristic Tuning:** Expanding and improving the compile-time heuristics and micro-optimizations across the framework.
- **Kernel Logic Redesign:** A structural redesign of the kernel logic to ensure seamless compatibility and high performance across all backends.
- **Hardware Expansion:** Compiling and testing the codebase on AMD GPUs (HIP), Intel architectures (SYCL), and embedded boards to fully validate the promise of Performance Portability.

6.2.15 A Complete Alpaka-Native Rewrite

Beyond incremental improvements, the most significant future work would be to throw everything away and rewrite MuDock natively and exclusively in Alpaka from day zero. This would result in a massive architectural "cleanup." Here is a breakdown of what would survive and what would be eliminated:

What would disappear:

- **90% of the backend folders:** All directories like `cuda_implementation/`, `hip_implementation/`, `syml_implementation/`, `omp_implementation/`, and `cpp_implementation/` would be completely deleted. There would be no need to maintain 6 different versions of the same kernel. Thousands of lines of duplicated code, prone to bugs and hard to maintain, would simply vanish.

- **Hardware-Specific Boilerplate:** Files like `cuda_utils.cuh`, `hip_utils.hpp`, error management macros (like `MUDOCK_CHECK`), and the manual management of streams and proprietary devices would disappear, completely absorbed internally by Alpaka's RAII framework.
- **CMake:** The `CMakeLists.txt` file would be drastically simplified, eliminating the need to manually manage compiler flags and search for CUDA toolkits, ROCm/HIP dependencies, or SYCL runtimes. . It would be enough to link the `alpaka::alpaka` target and let the library route to the correct system compiler.

What would remain:

- **The Chemical Domain and Parsing:** All the folders related to `src/chem/`, `src/format/`, and `src/molecule/`. The parsing of `.mol2` files, the OpenBabel integration, the Sybyl atomic types, and the biological logic of the algorithm would remain 100% intact. Alpaka abstracts hardware, not chemistry!
- **The Host Flow Graph:** The `tbb_implementation/` host pipeline that asynchronously routes ligands (read node → compute node → write node) would remain the perfect orchestrator. TBB manages task-based inter-ligand parallelism on the CPU (Macroscopic), while Alpaka is invoked inside the compute node to manage data-based intra-ligand parallelism on the GPU (Microscopic).
- **The Alpaka Files (The new "Truth"):** What is now called `alpaka_implementation/` would lose the suffix and simply become the standard `compute/` folder of the project. `adt_score_alpaka.cpp` would become the single, undisputed scoring function of MuDock.

Furthermore, having a single, universal Alpaka codebase would finally unlock one of the features of the library: the **Fat Binary**. Through template metaprogramming, the compiler can automatically instantiate the exact same kernel code for multiple backends (e.g., CPU OpenMP and NVIDIA CUDA) simultaneously within the same executable. The resulting binary performs Hardware Discovery at runtime: if it detects

a compatible GPU, it dispatches the workload to the accelerator; if no GPU is present, it seamlessly falls back to the heavily vectorized OpenMP CPU implementation.

6.2.16 Conclusions

The project has finally reached its conclusion. To avoid redundancy in an already extensive report, these final thoughts will be kept as brief as possible.

Overall, the work presented here has yielded extremely promising outcomes. As thoroughly detailed in the previous chapters, the original targets along with the supplementary ones introduced along the way fully met the performance expectations, definitively demonstrating the architectural potential of this framework.

However, an important final consideration must be made regarding the library's learning curve. As highlighted throughout this report, Alpaka is deeply abstracted and inherently complex, presenting a steep initial barrier to entry. Transitioning to this framework requires a paradigm shift in how a developer reasons about memory hierarchies, execution queues, and hardware-specific instructions. While this complexity might initially seem like a deterrent for adoption, it represents a highly favorable trade-off: it is undeniably more efficient to master the intricacies of one single, unified library than to learn, write, and maintain three or four entirely different proprietary paradigms (CUDA, HIP, SYCL, OpenMP) for every hardware platform on the market.

Alpaka has proven to be a mature and robust library, backed by solid support and a small but fiercely dedicated and growing community. On a personal note, its potential appears vast and still largely unexplored. The introduction of such hardware-agnostic paradigms in HPC is not only highly justified but increasingly necessary, making the prospect of a full, ground-up rewrite of MuDock an incredibly exciting future endeavor. To formally bring this report to a close, it seems fitting to recall the words of Edsger W. Dijkstra regarding the very nature of the abstraction work done here:

"The purpose of abstraction is not to be vague, but to create a new semantic level in which one can be absolutely precise."

References

- [1] Gianmarco Accordi and Davide Gadioli. muDock: A micro AutoDock-like implementation. GitHub repository, <https://github.com/elvispolimi/muDock>, 2024.
 - [2] Boost Community. Boost c++ libraries. <https://www.boost.org/>, 2024.
 - [3] M. Bussmann et al. Radiative signatures of the relativistic plasmonic photoelectric effect. *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC13)*, 2013.
 - [4] T. Gamblin, M. LeGendre, M. R. Collette, G. L. Lee, A. Moody, B. R. de Supinski, and S. Futral. The spack package manager: bringing order to hpc software chaos. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2015.
 - [5] M. Harris. Optimizing parallel reduction in cuda. <https://developer.download.nvidia.com/assets/cuda/files/reduction.pdf>, 2007.
 - [6] Kitware, Inc. Cmake reference documentation. <https://cmake.org/>, 2024.
 - [7] Makoto Matsumoto and Takuji Nishimura. Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation (TOMACS)*, 8(1):3–30, 1998.
 - [8] A. Matthes, R. Widera, E. Zenker, B. Worpitz, A. Huebl, and G. Juckeland. Tuning and optimization for a variety of many-core architectures without changing a single line of implementation code using the alpaka library. In *International Supercomputing Conference*, 2017.
 - [9] G. M. Morris, D. S. Goodsell, R. S. Halliday, R. Huey, W. E. Hart, R. K. Belew, and A. J. Olson. Automated docking using a lamarkian genetic algorithm and an empirical binding free energy function. *Journal of Computational Chemistry*, 19(14):1639–1662, 1998.
 - [10] G. M. Morris, R. Huey, W. Lindstrom, M. F. Sanner, R. K. Belew, D. S. Goodsell, and A. J. Olson. Autodock4 and autodock-tools4: Automated docking with selective receptor flexibility. *Journal of Computational Chemistry*, 30(16):2785–2791, 2009.
 - [11] NVIDIA Corporation. *CUDA C++ Programming Guide*, 2024. <https://docs.nvidia.com/cuda/>.
 - [12] N. M. O’Boyle, M. Banck, C. A. James, C. Morley, T. Vandermeersch, and G. R. Hutchison. Open babel: An open chemical toolbox. *Journal of Cheminformatics*, 3(1):33, 2011.
 - [13] OpenMP Architecture Review Board. *OpenMP Application Programming Interface Specification Version 5.2*, 2021.
 - [14] J. Reinders. *Intel Threading Building Blocks: Outfitting C++ for Multi-core Processor Parallelism*. O’Reilly Media, 2007.
 - [15] John K. Salmon, Mark A. Moraes, Ron O. Dror, and David E. Shaw. Parallel random numbers: as easy as 1, 2, 3. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–12. ACM, 2011.
-